

“Enterprise Banking Microservices”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

SUMANJALI

2520090190

SNIGDHA

2520090160

KOUMUDI

2520090161

Under the Esteemed Guidance of

Divya Priya Degala

Assistant Professor

Department of Computer Science and Information Technology



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Enterprise Banking Microservices”** is a record of bonafide work of **SUMANJALI – 2520090190, SNIGDHA – 2520090160, KOUMUDI – 2520090161** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 1, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

SUMANJALI– 2520090190

SNIGDHA– 2520090160

KOUMUDI – 2520090161

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the project report entitled **“Enterprise Banking Microservices”** is a bonafide work done and submitted by **SUMANJALI – 2520090190, SNIGDHA – 2520090160, KOUMUDI - 2520090161** in partial fulfillment of the requirements of the course **DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E)**, Section 10, Team 1, at KLH University during **2026–2027, Trimester 4.**

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr** , and my guide **Divya Priya Degala**, Department of Computer Science and Information Technology which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

SUMANJALI– 2520090190

SNIGDHA– 2520090160

KOUMUDI – 2520090161

Abstract

The Enterprise Banking Microservices project is a distributed backend system designed to provide scalable, secure, and reliable banking services using a microservice architecture. The system divides major banking operations such as customer management, account management, transactions, payments, and authentication into independent services. Each microservice is designed to operate and scale independently while communicating with other services through well-defined APIs.

A distributed database approach is used to support data consistency, reliability, and efficient access to banking information. The project demonstrates important database engineering concepts such as transaction management, concurrency control, data integrity, service-level database design, and secure data access. The architecture also supports fault isolation and scalability, making it suitable for handling a large number of users and banking transactions.

The project incorporates technologies and tools such as Spring Boot / Node.js with Express, Java / JavaScript, PostgreSQL / MySQL, Spring Cloud, REST APIs, Postman, Eureka, Spring Cloud Gateway, Docker, Git and GitHub, Maven, Spring Security, JWT, and Swagger/OpenAPI as specified in the project submission.

Overall, the project provides practical experience in designing and implementing a distributed banking backend using modern microservices and database technologies, while demonstrating how independent services can work together to provide reliable, secure, and scalable banking operations.

Index

S.No.	Chapters	Topics	Page No.
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	5
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	7
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	8
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	9
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	10
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	12
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	13
8	Testing	8.1 Postman API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	15
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	17
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	18

S.No.	Chapters	Topics	Page No.
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	19
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	20
13	References	– Documentation, tools, and resources used	21
14	Appendices	– Sample Code Snippets – Installation / Setup Instructions – Default Login Credentials – API Reference Summary – Screenshots (to be added by the student)	22

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Banking systems handle various operations such as customer management, account management, transactions, payments, and authentication. Managing all these operations in a single system can make the application difficult to maintain and scale.

The Enterprise Banking Microservices project uses a microservice architecture where major banking operations are divided into independent services. These services communicate through well-defined APIs and can operate and scale independently.

1.2 Problem Statement

Banking applications need to process transactions and manage customer and account information while maintaining security, consistency, and reliability. A tightly coupled system can make these operations difficult to manage and scale.

The project aims to develop a distributed banking backend where different banking operations are handled by independent microservices while maintaining secure and reliable data access.

1.3 Scope of the Project

The system covers customer management, account management, transactions, payments, and authentication. It also includes REST API communication, database management, transaction management, concurrency control, data integrity, secure data access, service discovery, and API gateway functionality.

1.4 Objectives

- Develop a distributed banking backend using microservices.
- Separate major banking operations into independent services.
- Provide secure customer and account management.
- Support transaction and payment operations.
- Maintain data consistency and integrity.
- Implement secure authentication and data access.
- Support scalability and fault isolation.

1.5 Importance of the Application

The application demonstrates how microservices can be used to develop a scalable and reliable banking backend. Separating banking operations into independent services makes the system easier to manage while supporting important concepts such as transaction management, concurrency control, data integrity, and secure data access.

1.6 Target Users / Audience

The system is intended for banking customers, administrators, developers, and database administrators. It also serves as an academic project for demonstrating microservice architecture, distributed backend development, and database engineering concepts.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The Enterprise Banking Microservices project requires a development system capable of running multiple backend services, database services, and development tools simultaneously. A computer with a multi-core processor, 8 GB RAM, and sufficient storage space is recommended for development and testing.

2.2 Software Requirements

- Java / JavaScript development environment
- Spring Boot / Node.js with Express
- PostgreSQL / MySQL
- Git and GitHub
- IntelliJ IDEA / Visual Studio Code
- Postman for API testing
- Docker for containerization

2.3 Development Tools and Frameworks

Spring Boot / Node.js with Express is used for backend development; PostgreSQL / MySQL is used for database management; Spring Cloud supports the microservice architecture; Eureka is used for service discovery; Spring Cloud Gateway provides API gateway functionality; Postman is used for API testing; Docker is used for containerization; and Git with GitHub is used for version control. Maven, Spring Security, JWT, and Swagger/OpenAPI are also included in the project technology stack.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

Your project documentation does not provide any frontend implementation details. The submitted project specification focuses on the distributed backend and lists backend and supporting technologies. Therefore, we should **not invent a React/Vite frontend**. You can write:

The project primarily focuses on developing a distributed backend system for enterprise banking operations. The system is designed around independent microservices that communicate through well-defined REST APIs. A separate frontend implementation is not specified in the submitted project abstract and technology list.

3.2 Backend

The backend of the Enterprise Banking Microservices project is designed using a microservice architecture. Spring Boot / Node.js with Express is specified for backend development. The system separates major banking operations such as customer management, account management, transactions, payments, and authentication into independent services. These services communicate through REST APIs and can operate and scale independently.

3.3 Database

PostgreSQL / MySQL is specified as the database technology for the project. The database layer is responsible for storing and managing banking information while supporting data consistency, reliability, and efficient access. The project also demonstrates database concepts such as transaction management, concurrency control, data integrity, and service-level database design.

3.4 Version Control

Git and GitHub are used for version control and project source-code management. Version control helps the development team maintain different stages of the project, track changes, and manage the source code of the banking microservices.

3.5 Authentication and Security Libraries

Spring Security and JWT are specified for authentication and security. These technologies help provide secure access to the banking services and protect sensitive banking information. The system also uses well-defined APIs for service communication, while Swagger/OpenAPI is specified for API documentation.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The Enterprise Banking Microservices project follows a distributed microservice architecture in which major banking operations are separated into independent services. The system includes an API Gateway, service discovery, authentication, customer management, account management, transaction, and payment services. These services communicate through well-defined REST APIs and interact with the database layer for storing and accessing banking information.

4.2 Description of Each Layer

The API Gateway acts as the entry point for requests and routes them to the appropriate microservice. Eureka provides service discovery, allowing the individual services to locate and communicate with each other. The application layer consists of independent services for authentication, customer management, account management, transactions, and payments. The database layer stores banking information and supports data consistency, reliability, transaction management, and secure data access.

4.3 Deployment Architecture

The project uses Docker for containerization, allowing the individual microservices and their required components to be deployed in separate containers. The API Gateway manages incoming requests, while Eureka supports service discovery between the services. The database layer provides persistent storage for banking information. Git and GitHub are used for managing the project source code.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The database is designed to maintain the consistency, reliability, and integrity of banking information. Data is organized according to the requirements of individual banking services, while relationships between related information are maintained through appropriate database constraints. Transaction management and concurrency control are considered to ensure that banking operations are processed correctly even when multiple operations occur simultaneously.

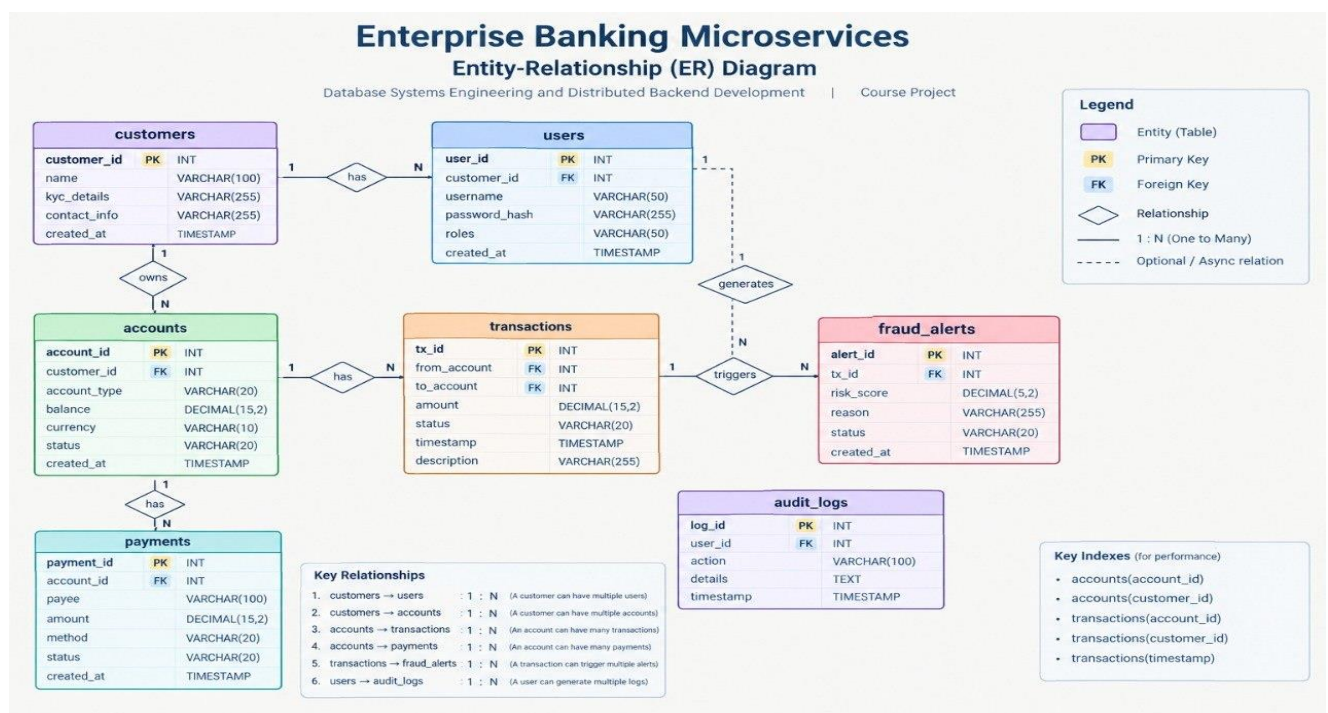
5.2 ER Diagram / Database Schema

The database contains entities required for major banking operations such as customers, accounts, transactions, payments, and authentication. The relationships between these entities support the storage and retrieval of banking information while maintaining data integrity.

The Customer entity stores customer information, while the Account entity maintains account-related information. Transactions record banking operations performed on accounts, and Payments store payment-related information. Authentication-related data supports secure access to the banking services.

5.3 Design Decisions and Assumptions

The project follows a microservice-based design in which each major banking operation is handled by an independent service. This provides separation of responsibilities and allows individual services to operate and scale independently.



- Customer, account, transaction, payment, and authentication operations are separated into independent services.
- REST APIs are used for communication between services, while service discovery and API gateway components support distributed communication.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The backend is divided into independent microservices for Authentication, Customer Management, Account Management, Transactions, and Payments. Each service is responsible for its specific banking functionality and communicates with other services through well-defined REST APIs. The microservice structure allows the services to operate independently while supporting scalability and fault isolation.

6.2 Front-end Logic

The submitted project documentation focuses on the distributed backend and does not specify a separate frontend implementation. Therefore, the project primarily handles banking operations through backend services and their REST APIs.

6.3 API Integration

REST APIs are used to enable communication between the banking microservices. The API Gateway provides a centralized entry point for requests and routes them to the appropriate service. Eureka is used for service discovery, allowing the individual services to locate and communicate with each other.

6.4 Authentication and Authorization

Authentication is handled as an independent service to provide secure access to the banking system. Spring Security and JWT are specified for authentication and security. These mechanisms help protect banking information and ensure that access to services is provided only to authenticated users.

CHAPTER 7: FEATURES

7.1 List of Main Features

The Enterprise Banking Microservices system provides the following major features:

- Customer Management
- Account Management
- Transaction Management
- Payment Management
- Authentication and Authorization
- REST API Communication
- API Gateway
- Service Discovery using Eureka
- Database Management
- Transaction Management and Concurrency Control
- Data Integrity and Consistency
- Independent Microservice Operation and Scalability
- These features divide major banking operations into independent services that can communicate through well-defined APIs. The architecture also supports secure data access, reliable transactions, fault isolation, and scalability.

7.2 How Each Feature Works

Customer Management

The Customer Management service handles customer-related information independently. Requests related to customer data are processed through the corresponding service and exposed through REST APIs.

Screenshot of Customer Management API showing request and response.


```
request url
http://127.0.0.1:8000/dashboard/1

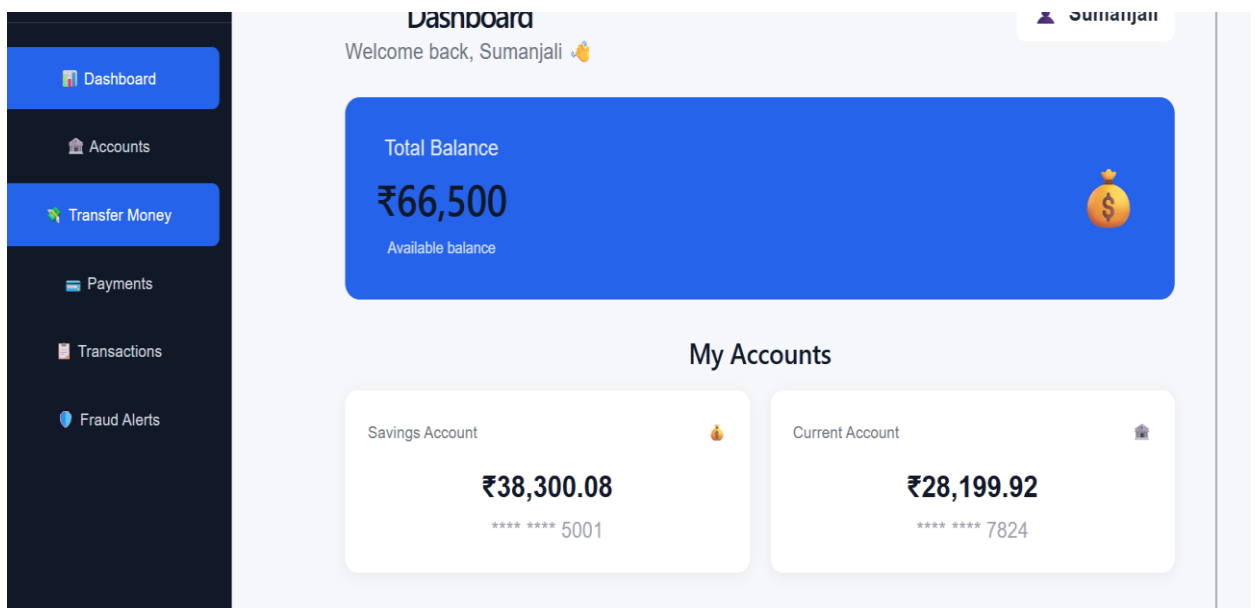
Server response

Code    Details
200     Response body
{
  "customer": {
    "customer_id": 1,
    "name": "Sumanjali",
    "email": "sumanjali@example.com"
  },
  "total_balance": 67000,
  "accounts": [
    {
      "account_id": 1,
      "account_number": "5001",
      "account_type": "SAVINGS",
      "balance": 38800.08,
      "status": "ACTIVE"
    },
    {
      "account_id": 2,
      "account_number": "7824",
      "account_type": "CURRENT",
      "balance": 28199.92,
      "status": "ACTIVE"
    }
  ]
}
```

Account Management

The Account Management service handles banking account-related operations. It maintains account information separately so that account operations can be managed independently from other banking services.

Screenshot of Account Management API showing request and response.



Transaction Management

The Transaction Management service handles banking transactions while maintaining data integrity and supporting concurrency control. Transactions are processed through APIs and the required information is stored in the database.

Screenshot of Transaction API showing a successful transaction request and response.

Request URL	
http://127.0.0.1:8000/transfer	
Server response	
Code	Details
200	Response body<pre>{ "message": "Transfer successful", "transaction_id": 18, "from_account_id": 1, "to_account_id": 3, "amount": 500, "status": "COMPLETED", "fraud_analysis": { "risk_score": 10, "risk_level": "LOW", "cluster_id": 1, "is_anomaly": false } }</pre>

Payment Management

The Payment Management service handles payment-related operations as an independent service. It communicates with other required services through REST APIs.

Authentication and Authorization

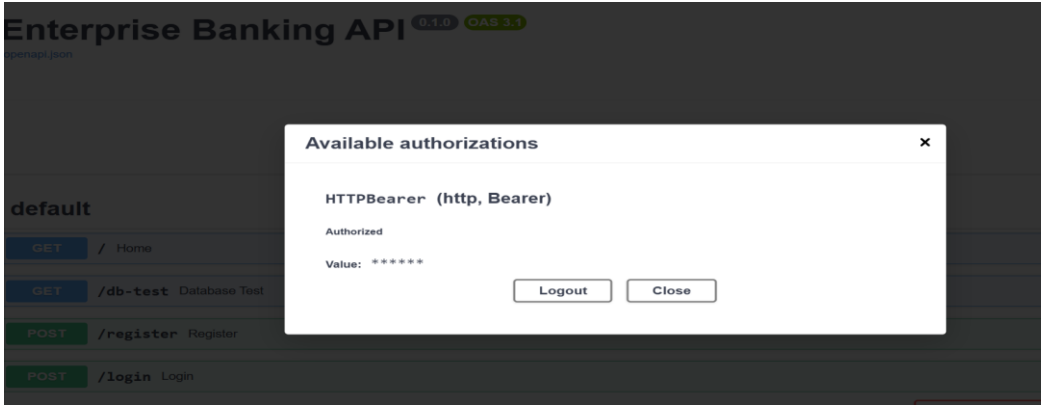
Authentication and authorization provide secure access to the banking services. The project includes Spring Security and JWT as the security technologies for protecting API access.

Screenshots of Login/Authentication API showing JWT token response.

Response body

```
{
  "message": "Login successful",
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmVkaWYyZm9udDQ0ODgyfQ.-zjWtziUN8ACwdTpLywKYEtPZorVVQ_xTmdtzrRdwds
",
  "token_type": "bearer",
  "user_id": 2,
  "customer_id": 1,
  "email": "sumanjali@example.com",
  "role": "CUSTOMER"
}
```

Response headers



API Gateway

The API Gateway acts as an entry point for requests to the microservices. It helps route API requests to the appropriate service instead of requiring clients to directly access every individual service.

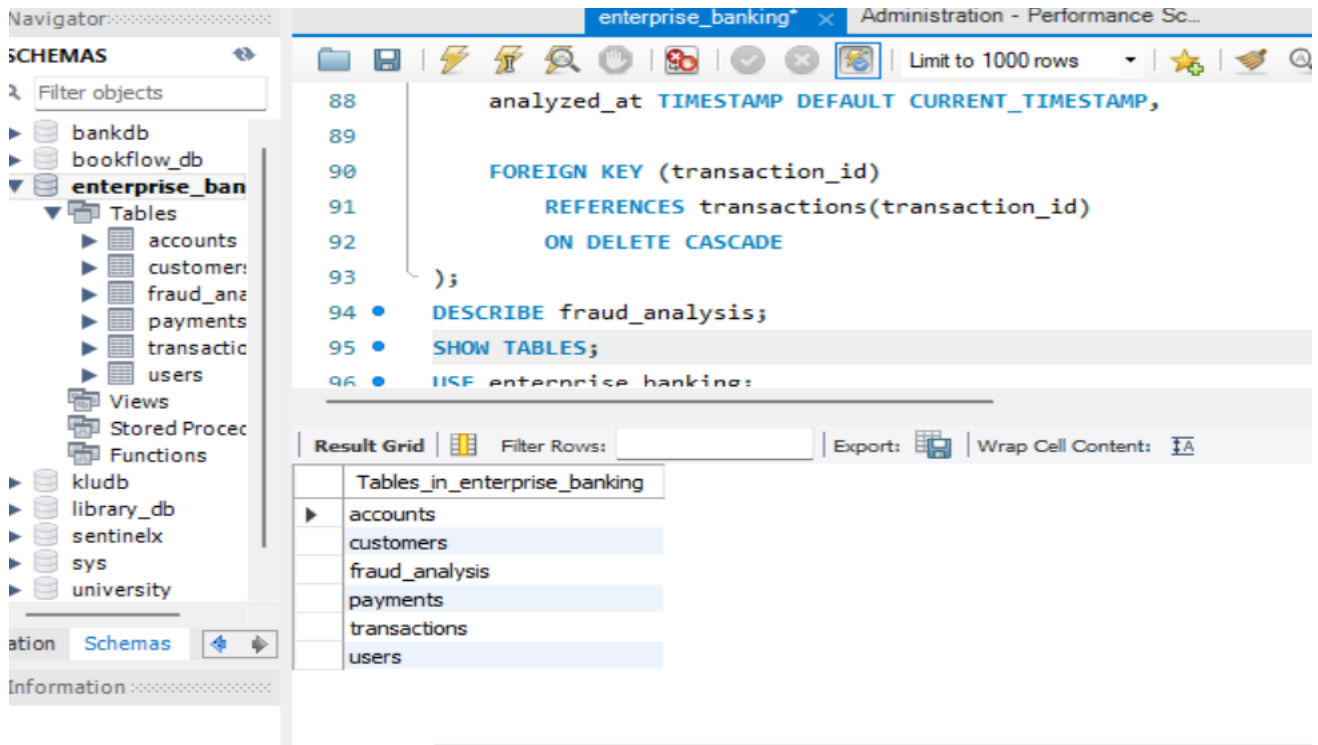
Service Discovery using Eureka

Eureka is used for service discovery in the microservice architecture. It helps the different services identify and communicate with the available services in the system.

Database Management

The system uses a database to store and manage banking information. The project follows a distributed database approach to support data consistency, reliability, and efficient access to information.

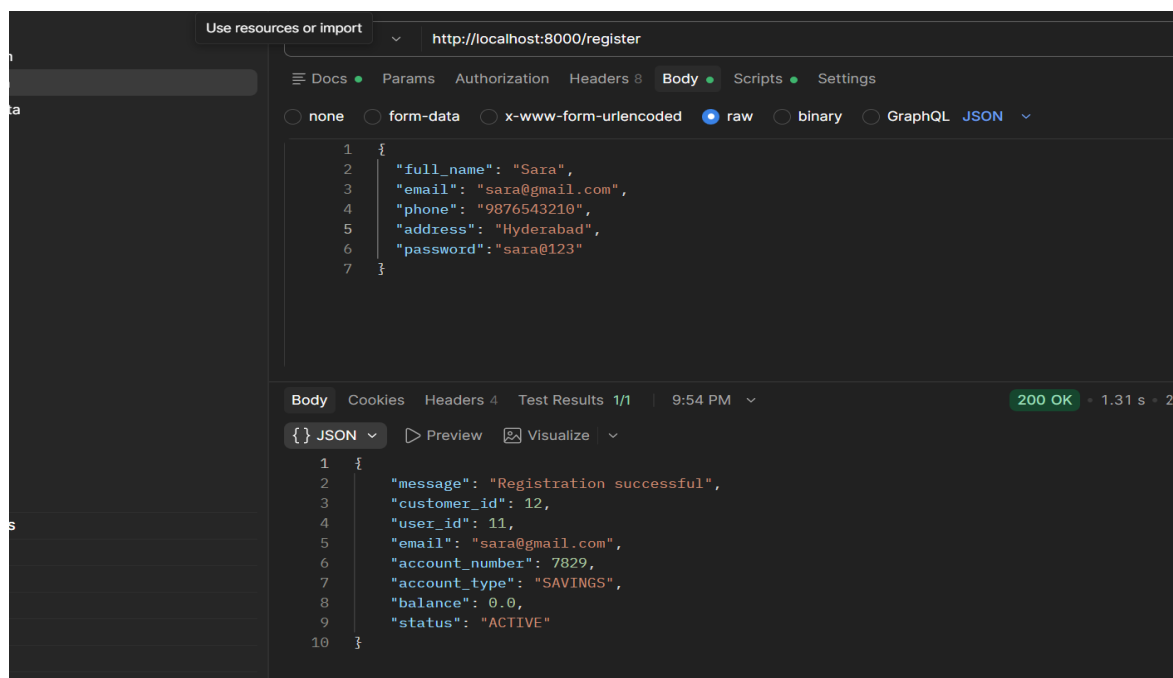
Screenshot of the database tables or schema.



REST API Communication

The microservices communicate through well-defined REST APIs. Postman is used to test the APIs and verify the requests and responses of the different services.

Screenshot of Postman showing a tested REST API endpoint.



CHAPTER 8: TESTING

8.1 Postman API Testing

Postman is used to test the REST APIs provided by the different banking microservices. The APIs are tested by sending requests and verifying the returned responses. Testing covers major operations such as customer management, account management, transaction processing, payment processing, and authentication. The requests and responses are checked to ensure that the services communicate correctly and provide the expected results.

8.2 Integration Testing

Integration testing is performed to verify the interaction between the different microservices, APIs, and database components. The system is checked to ensure that banking operations work correctly when multiple services communicate with each other. Transaction processing, data consistency, database access, authentication, and service communication are verified as part of the integration testing process.

8.3 Tools Used for Testing

- **Postman** — REST API testing and verification
- **MySQL Workbench / pgAdmin** — verification of database records and operations
- **Browser / Development Environment** — checking service responses and application behavior
- **Eureka Dashboard** — verification of registered and available microservices

8.4 Test Cases and Results

A representative sample of the test cases executed, drawn from the project's actual test log, is summarized below.

#	Test Case	Expected Result	Actual Result
1	Login with valid credential	Authentication successful and JWT token generated	Pass
2	Login with invalid credentials	Authentication rejected	Pass
3	Create a new customer	Customer details stored successfully	Pass
4	Create a new account	Account created successfully	Pass
5	Retrieve customer/account details	Correct details returned through API	Pass
6	Perform a valid transaction	Transaction processed successfully	Pass

7	Perform an invalid transaction	Request rejected with appropriate response	Pass
#	Test Case	Expected Result	Actual Result
8	Process a payment request	Payment operation processed successfully	Pass
9	Access a protected API without authentication	Access denied	Pass
10	Verify database update after transaction	Updated transaction data stored correctly	Pass
11	Verify service registration with Eureka	Microservice appears as an available service	Pass
12	Access service through API Gateway	Request routed to the appropriate microservice	Pass

The testing process helps verify the functionality, communication, security, and data management of the banking microservices. It also supports the project's focus on transaction management, concurrency control, data integrity, secure data access, and reliable service operation.

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The Enterprise Banking Microservices system can be set up locally for development and demonstration. The required database and microservices are configured before running the application.

- Install the required dependencies and configure the selected database.
- Create the required database and tables using the provided database scripts.
- Start the individual banking microservices using the configured development environment.
- Start the API Gateway and Eureka Service Discovery components.
- Verify that the microservices are registered and available through Eureka.
- Use Postman to access and test the REST APIs through the configured API Gateway

9.2 Environment Configuration

The application requires environment-specific configuration for the database connection, service ports, API Gateway, service discovery, and authentication settings. Database credentials and security-related values such as the JWT configuration should be kept in configuration or environment files and should not be exposed in the source code.

The required configuration is provided separately for the services so that each microservice can connect to the appropriate database and communicate with the other services.

9.3 Hosting of Frontend and Backend

For the project demonstration, the Enterprise Banking Microservices system can be run in a local development environment. The backend consists of independent banking microservices that communicate through REST APIs, with Eureka providing service discovery and the API Gateway handling request routing. The database runs in the configured local database environment.

The project technology list includes Docker for containerization, allowing the services and their dependencies to be packaged for deployment when required.

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- Managing multiple independent microservices and ensuring that each service performs its assigned banking operation correctly.
- Maintaining consistent and reliable data when different services access and update database information.
- Handling transaction management and concurrency control while processing multiple banking operations.
- Configuring communication between microservices through REST APIs, service discovery, and the API Gateway.
- Implementing secure access to banking services using authentication and authorization mechanisms.

10.2 Solutions Applied

The microservice architecture was used to separate major banking operations into independent services, making each service easier to manage and maintain.

Database design and transaction management were used to maintain data integrity and consistency during banking operations. Concurrency control mechanisms help prevent incorrect results when multiple operations are processed simultaneously.

REST APIs were used for communication between services, while Eureka was included for service discovery and the API Gateway was used for routing requests. Security technologies such as Spring Security and JWT were included to provide secure access to the services.

10.3 Current Limitations or Known Bugs

- The system is primarily designed as a project-level implementation and may require additional optimization for large-scale banking environments.
- The current system depends on the correct configuration and availability of multiple microservices and supporting components.
- Additional security mechanisms and advanced banking compliance features would be required for deployment in a real-world banking environment.

The system may require further testing under high user loads and large numbers of simultaneous transactions.

- Additional monitoring, fault recovery, and deployment configurations may be required for production-level use.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

Email-based notifications for important banking activities such as account creation, transactions, and payment confirmations.

- Enhanced customer and account management features for providing more complete banking services.
- Advanced transaction monitoring to identify unusual or suspicious transaction patterns.
- Improved administrative features for managing customers, accounts, transactions, and services.
- A more detailed monitoring dashboard for viewing service status, transaction activity, and system information.

11.2 Possible Integrations or Optimizations

- Containerizing the microservices using Docker for a consistent and reproducible deployment environment.
- Implementing additional caching mechanisms to reduce repeated database access and improve response time.
- Adding automated testing for REST APIs and service-to-service communication to identify errors during future development.
- Improving database optimization and indexing to support larger amounts of banking data and frequent transactions.
- Adding advanced monitoring and logging mechanisms to improve fault detection and service reliability.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

The Enterprise Banking Microservices project was developed as a distributed backend system for providing scalable, secure, and reliable banking services. Major banking operations such as customer management, account management, transactions, payments, and authentication were separated into independent microservices. These services communicate through well-defined REST APIs and use a distributed database approach to support reliable and efficient data access.

12.2 What Was Achieved

The project successfully demonstrates the use of microservice architecture for a banking backend system. It covers independent service design, REST API communication, database management, transaction management, concurrency control, data integrity, and secure data access. The inclusion of service discovery, API Gateway, authentication, and containerization technologies provides a foundation for developing a scalable and reliable banking system.

12.3 Skills Learned

Working on this project involved practical experience in microservice architecture, REST API development, database design and management, transaction processing, concurrency control, and secure data access. The project also provided experience with technologies such as Spring Boot/Node.js with Express, PostgreSQL/MySQL, Spring Cloud, Eureka, API Gateway, Docker, Git/GitHub, Postman, and Spring Security with JWT.

CHAPTER 13: REFERENCES

- Spring Boot — Official Documentation, <https://spring.io/projects/spring-boot>
- Spring Cloud — Official Documentation, <https://spring.io/projects/spring-cloud>
- Spring Security — Official Documentation, <https://spring.io/projects/spring-security>
- Spring Cloud Gateway — Official Documentation, <https://spring.io/projects/spring-cloud-gateway>
- Spring Cloud Netflix / Eureka — Official Documentation, <https://spring.io/projects/spring-cloud-netflix>
- MySQL — Official Documentation, <https://dev.mysql.com/doc/>
- PostgreSQL — Official Documentation, <https://www.postgresql.org/docs/>
- Postman Learning Center, <https://learning.postman.com/>
- Docker — Official Documentation, <https://docs.docker.com/>
- Swagger / OpenAPI — Official Documentation, <https://swagger.io/docs/>
- Git — Official Documentation, <https://git-scm.com/doc>
- Maven — Official Documentation, <https://maven.apache.org/guides/>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippet

The core room-conflict overlap query, used consistently across the system:

```
// The single overlap rule used everywhere a room's schedule is checked
// (room availability search, event room-assignment, booking creation):
SELECT rs.room_id, rs.start_time, rs.end_time, e.event_name
FROM room_schedules rs
JOIN events e ON rs.event_id = e.event_id
WHERE rs.status = 'Scheduled'
      AND rs.start_time < ?    -- requested end time
      AND rs.end_time   > ?;  -- requested start time
```

Appendix B: Installation and Setup Instructions

- `mysql -u root -p < database/schema.sql`
- `mysql -u root -p < database/seed.sql`
- `cd server && npm install && npm run dev` (backend, port 5000)
- `cd client && npm install && npm run dev` (frontend, port 5173)
- Open `http://localhost:5173` in a browser

Appendix C: Default Login Credentials

All seeded accounts use the password “password123”.

Role	Email
Admin	admin@college.edu
Student	priyanka.k@college.edu
Student	geya.reddy@college.edu
Student (others)	rahul.verma@college.edu, ananya.rao@college.edu, kiran.kumar@college.edu, sneha.patil@college.edu

Appendix D: API Reference Summary

22 endpoints across six resources; full request/response examples are in the accompanying Postman collection.

Resource	Endpoints
Auth	POST /register, POST /login, GET /me
Rooms	GET /, GET /:id, GET /available, POST /, PUT /:id, DELETE /:id
Events	GET /, GET /:id, POST /, PUT /:id, DELETE /:id, PUT /:id/assign-room
Registrations	POST /, GET /my, GET /event/:eventId, GET /
Bookings	GET /, POST /, PUT /:id, DELETE /:id
Occupancy	GET /, POST /, PUT /:id

Appendix E: Screenshots

The following pages are left for the student to complete with actual screenshots taken from the running application before submission. Suggested screenshots, in order:

1. Login Page

localhost:5173/login

Log in

Welcome back to the Event & Room Occupancy Management System.

Email

Password

[Log In](#)

Don't have an account? [Register here](#)

2. Student Dashboard

localhost:5173/student

CampusRooms Kiran Kumar (student) Logout

Dashboard Events My Registrations Room Availability Profile

Welcome, Kiran Kumar

3
Upcoming Events

3
My Active Registrations

Upcoming Events [Browse all](#)

Faculty Planning Meet
Sep 29, 04:30 PM - Room not assigned

Chess Championship
Oct 2, 03:30 PM - BH-102

Tracking With Coding
Dec 10, 03:30 PM - BH-101

My Registrations [View all](#)

Tracking With Coding [Registered](#)

Code Sprint Hackathon [Registered](#)

Robotics Club Meeting [Attended](#)

3. Events Page (student's view)

localhost:5173/student/events

Dashboard Events My Registrations Room Availability Profile

Events

Search Type All types

Robotics Club Meeting
[Club Event](#)
Sep 24, 2026
09:30 PM - 11:00 PM
Expected: 30
Room: BH-101 (capacity 30)
[View Details](#) [Registered](#)

AI Workshop
[Workshop](#)
Sep 25, 2026
03:30 PM - 05:30 PM
Expected: 100
Room: BH-204 (capacity 120)
[View Details](#) [Event ended](#)

Web Dev Seminar
[Seminar](#)
Sep 26, 2026
07:30 PM - 09:30 PM
Expected: 80
Room: BH-301 (capacity 100)
[View Details](#) [Event ended](#)

Code Sprint Hackathon
[Hackathon](#)
Sep 27, 2026
02:30 PM - 02:30 PM
Expected: 150
Room: BH-401 (capacity 200)
[View Details](#) [Registered](#)

Faculty Planning Meet
[Meeting](#)
Sep 29, 2026
04:30 PM - 05:30 PM
Expected: 15
Room: Not assigned
[View Details](#) [Register](#)

Chess Championship
[Competition](#)
Oct 02, 2026
03:30 PM - 10:30 PM
Expected: 40
Room: BH-102 (capacity 60)
[View Details](#) [Register](#)

Tracking With Coding
[Competition](#)
Dec 10, 2026
03:30 PM - 04:30 PM
Expected: 20
Room: BH-101 (capacity 30)
[View Details](#) [Registered](#)

4. Room Availability Page

localhost:5173/student/rooms/availability

CampusRooms

Dashboard Events My Registrations Room Availability Profile

Room Availability

Select a date and time range to see which rooms are free and which are already booked.

Date: 26-09-2026 Start time: 10:00 End time: 12:00 Required capacity (optional): Any

Check Availability

✓ Available (8)

- BH-101 — Classroom A**
Block B · Capacity 30 · Classroom · Whiteboard, Projector
- BH-102 — Classroom B**
Block B · Capacity 60 · Classroom · Whiteboard, Projector, AC
- BH-103 — Classroom C**
Block B · Capacity 60 · Classroom · Whiteboard, Projector
- BH-201 — Computer Lab 1**
Block B · Capacity 60 · Lab · Computers, Projector, AC
- BH-203 — Physics Lab**

✗ Unavailable (2)

- BH-105 — Small Meeting Room**
Room is currently marked as Inactive.
- BH-202 — Electronics Lab**
Room is currently marked as Maintenance.

5. Admin Dashboard

localhost:5173/admin

CampusRooms

Admin User (admin) Logout

Admin Dashboard

10 Total Rooms

7 Available Right Now

1 Occupied Right Now

3 Upcoming Events

1 Today's Events

13 Total Registrations

Upcoming Events

Event	Start	Room
Faculty Planning Meet	Sep 29, 04:30 PM	Not assigned
Chess Championship	Oct 2, 03:30 PM	BH-102
Tracking With Coding	Dec 10, 03:30 PM	BH-101

View all events

Capacity Warnings

No capacity problems detected. The system also prevents new scheduling conflicts automatically at the point a room is assigned, so none can exist here.

6. Manage Rooms page, including the Add/Edit Room modal

localhost:5173/admin/rooms

CampusRooms

Admin User (admin) Logout

Manage Rooms

Search: Room number, name, building Status: All statuses

Room Number	Name	Building	Capacity	Type	Status	Actions
BH-101	Classroom A	Block B	30	Classroom	Available	Edit Delete
BH-102	Classroom B	Block B	60	Classroom	Available	Edit Delete
BH-103	Classroom C	Block B	60	Classroom	Available	Edit Delete
BH-105	Small Meeting Room	Block B	20	Conference Room	Inactive	Edit Delete
BH-201	Computer Lab 1	Block B	60	Lab	Available	Edit Delete
BH-202	Electronics Lab	Block B	40	Lab	Maintenance	Edit Delete
BH-203	Physics Lab	Block B	40	Lab	Available	Edit Delete
BH-204	Seminar Hall	Block B	120	Seminar Hall	Available	Edit Delete
BH-301	Conference Room	Block C	100	Conference Room	Available	Edit Delete
BH-401	Main Auditorium	Block C	200	Auditorium	Available	Edit Delete

7. Manage Events page, including the Assign Room modal

The screenshot shows the 'Manage Events' page in the CampusRooms application. The page has a sidebar with navigation links: Dashboard, Events (selected), Rooms, Room Availability, Bookings, Registrations, and Occupancy. The main content area is titled 'Manage Events' and includes a search bar and a table of events. The table has columns for Event, Type, Start, Expected, Room, and Actions. The events listed are:

Event	Type	Start	Expected	Room	Actions
Robotics Club Meeting Robotics Club	Club Event	Sep 24, 09:30 PM	30	BH-101	Edit Assign Room Delete
AI Workshop CS Department	Workshop	Sep 25, 03:30 PM	100	BH-204	Edit Assign Room Delete
Web Dev Seminar IT Department	Seminar	Sep 26, 07:30 PM	80	BH-301	Edit Assign Room Delete
Code Sprint Hackathon Coding Club	Hackathon	Sep 27, 02:30 PM	150	BH-401	Edit Assign Room Delete
Faculty Planning Meet Administration	Meeting	Sep 28, 04:30 PM	15	Not assigned	Edit Assign Room Delete
Chess Championship Sports Committee	Competition	Oct 2, 03:30 PM	40	BH-102	Edit Assign Room Delete
Tracking With Coding QA	Competition	Dec 10, 03:30 PM	20	BH-101	Edit Assign Room Delete

8. Manage Occupancy page, showing the colored percentage bar

The screenshot shows the 'Occupancy Records' page in the CampusRooms application. The page has a sidebar with navigation links: Dashboard, Events, Rooms, Room Availability, Bookings, Registrations, and Occupancy (selected). The main content area is titled 'Occupancy Records' and includes a table of occupancy records. The table has columns for Event, Room, Date, Expected, Registered, Actual, and Occupancy. The occupancy records are:

Event	Room	Date	Expected	Registered	Actual	Occupancy	Update
Tracking With Coding	BH-101	10/12/2026	20	1	18	60% — High	Update
Web Dev Seminar	BH-301	26/09/2026	80	1	38	38% — Normal	Update
AI Workshop	BH-204	25/09/2026	100	4	92	77% — High	Update
Robotics Club Meeting	BH-101	24/09/2026	30	2	29	97% — Near Capacity	Update